

Blockchain-Based Secure Data Sharing Device

Shivam Mittal¹, Gaurav Negi², Divyam Dhenwal³ and Dimple Tiwari^{4*}

School of Engineering & Technology, Vivekananda Institute of Professional Studies - Technical Campus, Delhi 110034, India.

Corresponding Email: 06917702722_cse@vips.edu, 10217702722_cse@vips.edu, 08617702722_cse@vips.edu, dimple.tiwari88@gmail.com

Abstract— As digital file sharing continues to grow, users need platforms that are not only fast and convenient but also secure and reliable. Many existing systems face challenges such as weak access control, limited scalability, and lack of trust in file integrity. We proposed a modern secure file sharing platform designed to solve these issues through an integrated multi-service system. In this system we proposed a web application for browser users, a command-line interface for technical users, a backend service for authentication and file management, and a machine learning module for risk detection. Instead of routing large files through the server, SHC uses presigned cloud storage URLs, which improves performance and reduces server load. It also provides features such as OTP-based login, JWT authentication, file visibility control, automatic file expiry, and secure sharing options. It also uses blockchain-based hash notarization to verify file integrity and machine. SHC has a lot of good security features like a code to log in a way to authenticate users control over who can see files files that automatically expire and secure ways to share files. To make sure files are safe from start to finish SHC uses something called blockchain to keep track of the files on the Ethereum Sepolia testnet. This means that if someone tries to tamper with a file it will be obvious to everyone. The machine learning part of SHC uses a combination of models to figure out how risky a shared link is. It uses something called Shannon entropy and SHAP to make sure the risk decisions are clear and understandable.

I. Introduction

Over the past decade, research communities around the world have been working on efficient and more secure methods of data sharing. Data Sharing enables meaningful collaborations and informed decision making – essential for advanced research innovation. All the experiments done throughout, a basic data sharing process must answer fundamental questions – what data will be shared, when it will be shared, where will it go. Shrestha and Vassileva[1] were among the first to highlight this gap, pointing that meaningful incentives for data owners are essential to making research data sharing sustainable in long run. How data owners get fairly compensated remains an open and largely ignored problem. This research tackles it head on by proposing a blockchain-based framework for secure data sharing that keeps the owner in control of access, of distribution, and of payment. Blockchain is a distributed ledger maintained across multiple nodes rather than any single server. That dependency creates privacy risks, security vulnerabilities, and single points of failure. Nakamoto challenged this in 2008 with Bitcoin [2], proving that two parties could transact directly with no intermediary at all. That idea changed everything. Since then, blockchain has found its way into cloud computing, IoT, healthcare, data trading, and information security [3] — anywhere trust matters but centralized control causes more problems than it solves. Research data sharing is one of those places. The biggest reason researchers avoid sharing their data is fear — fear of misuse, misinterpretation, and losing control once it leaves their hands.

A. Research Gap

Loss of Visibility After Sharing: When you share data you lose sight of it. You do not know who is looking at it. You also do not know how they are using it. You even do not know if they are getting any value, from it. The data owner has no visibility. They

cannot see who accesses the data. They also do not know how the data is being used. The owner also does not know if they are getting anything back.

Lack of Trust and Immutability: People do not trust things that're not trustworthy. Researchers need to feel safe when they share their data. The things that make researchers feel comfortable sharing their data are trust and immutability and traceability of the data they are sharing. We rarely see all of these things in one place like in one solution, for data sharing.

No Transparency Without a Third Party: Of thinking of another way to control who can see things or keep information secret the focus here is on using blockchain and decentralized storage together. So everything about transactions is clear and people get paid for what they do, without needing someone else like a third party to make it happen.

B. Contribution

Owner Controlled Access: We made sure that the people who own the data can still control their files even after they put them up. They can decide who can see and use their files. This is done through a way of checking who is allowed to access the files called JSON Web Tokens and by using a button to turn visibility on or off. This means that nobody can move or look at a file without the owner's permission. The owner has to say it is okay, for someone to access or move their file.

On-Chain File Integrity: We made a way for people to check if a shared file is still the same after it was uploaded. This means you do not have to trust the website where the file is stored. We do this by adding the file's SHA-256 hash to the Ethereum Sepolia testnet. This creates a record that will always be there and anyone can check it on their own to make sure the file is still the Ethereum Sepolia testnet record.

Public Integrity Verification Endpoint: We made it simple for people to verify that a file is genuine. They can do this without having to log in or rely on our platform. We have an endpoint that anyone can use to calculate the file's SHA-256 and then compare it to the data that is stored on the blockchain. This way people can check the file integrity themselves.

This paper is set up in the way. The second part of this paper talks about what other people have done in the area of cloud file sharing and things like using codes storing things on blockchain controlling who gets to access things finding threats and making sure people do not use too much space. It also has a table that summarizes everything that is mentioned. The third part of this paper explains how we did our work, including what our system looks like how we got our data how we prepared it how we trained our models and how we set up our experiments. The fourth part of this paper shows and talks about what we found out including how our system can detect risks how long it takes for things to happen which features are the most important, what we learned from using blockchain and how our system compares to other systems. The last part of this paper, which's the fifth part summarizes everything and talks about what we can do, in the future.

II. LITERATURE SURVEY

File Sharing and Cloud Storage The popular file sharing platforms today like Google Drive, Dropbox and OneDrive work in a similar way. You upload a file it goes through their server gets stored somewhere and when someone downloads it it comes back through their server. This approach has some drawbacks.

The server can become a bottleneck when many people are uploading once. It can also become a point of failure if something goes wrong. As file sizes and user counts grow it becomes a cost. Using presigned URLs to bypass these problems is not an idea. Cloud providers like AWS S3 and Cloudflare R2 have supported it for years. However no file sharing platform has made this a key part of their design. Most platforms still use their backend to proxy transfers because it is simpler to build. With all the good things about presigned URLs no big file sharing platform uses them as the main way to transfer files.

With our research it came to our notice that majority platforms still use their servers to handle all file transfers. This is because it makes it easier to control who can access files and it is simpler for developers to set up. This easy setup comes with a cost later on. As files get bigger and more people use the platform it costs more to run the servers. The people who run the platform have to pay for every gigabyte of bandwidth that goes through their servers and they have to pay for it twice once when it comes in and once when it goes out. For platforms that have millions of users uploading video files, data or software this becomes a very big cost that affects how much they charge and how they can grow. SHC was built from the start to use presigned URLs and to transfer files directly to storage not as something to add later.

As a main part of how it works, SHC uses the presigned URL model and direct-to-storage transfer as a core part of its architecture, which's different, from other platforms.

This creates problems when dealing with large numbers of users. **Secure File Storage and Sharing on Cloud Using Cryptography** Madhumala et al.[21] proposed a way to store and share files on the cloud that's safe and secure. They used cryptography to protect the files people upload. This means that the files are encrypted before they are stored on servers that are not controlled by the people who uploaded them. This makes it harder for people to get to the files without permission. It also makes people trust the cloud more. It shows how important encryption is when people share files. However their system still relies on companies that control the cloud. This means that there are still problems like what happens if the system fails or if people do not trust the companies. It also does not deal with making sure that the files are safe and that people can see what is happening to them.

Secure Data Sharing Platform Using Blockchain and InterPlanetary File System Naz et al.[22] proposed a way to share data that's safe and secure. They used blockchain and the InterPlanetary File System. The blockchain is like a book that keeps track of everything that happens. It cannot be changed. The InterPlanetary File System is a way to store files on different computers at the same time. This makes the system more safe and reliable, than cloud systems. It shows how blockchain and the InterPlanetary File System can work together to make sharing data more safe. However they mostly focused on how to keep the data safe and how to share it. They did not think about things that people need to use the system like a way to log in or a way to stop bad people from sharing bad links.

The idea of using blockchain to create tamper-evident records is useful. Wood et al.[9] introducing Ethereum and smart contracts made this possible. Nizamuddin et al. [11] have used Ethereum and IPFS together to track document versions on a blockchain. Hasan and Salah et al.[12] have shown that proof of delivery for assets can be verified using smart contracts. These solutions do

not go far enough. Panescu and Manta et al. [13] applied this specifically to research data rights on Ethereum. They do not provide an endpoint that anyone can use to verify that a specific file has not been changed since it was uploaded. This is a gap. If a platform says a file is notarized it should be possible to verify this without going through the platform.

Benet et al.in [4] introduced Decentralized Storage IPFS is a peer-to-peer alternative to file storage. Of fetching a file from a specific server IPFS addresses content by what it is. This means that even if individual nodes go offline the data remains available elsewhere on the network. Chen et al.in [14] proposed to have combined IPFS with blockchain to build resilient P2P file systems. Steichen et al.in [15] have added access control on top of IPFS using blockchain. These are contributions but they are research prototypes, not complete platforms. They do not provide a user- interface or handle tasks like user authentication and quota enforcement Access. Authentication Wu et al. [4] have integrated attribute-based encryption into blockchain to make data access more secure. Sun et al. [16] gave owners medium to control who can access their data and provides a way to track where a secret key came from. These solutions are complex to implement and use. They do not provide an authentication flow that works across different devices. A developer logging in through a command line interface needs the session and permissions as someone using a web interface.

Threat Detection on Shared Links There is a gap in the literature when it comes to evaluating the risk of a shared file link. Park et al.[18] security research focuses on endpoint detection scanning files after they have already been downloaded and executed. This is too late. What if a platform could assess the risk of a shared file link before it was downloaded ?What if every shared link was evaluated for malware and phishing signals in time and the result was shown to the person before they clicked download? This research aims to fill this gap.Quota Enforcement and Multi-Tenancy Most file sharing systems. Have no usage limits or implement them in ways that are easy to work around. A proper multi-tenant system needs limits, on reads writes and storage enforced at the backend level introduced by Li&Zhu et al.[20]. Without this a single user can consume resources meant for everyone and the whole system becomes unpredictable at scale. This is largely ignored in existing work.

Most file sharing systems do not have any rules about how you can use them or they have rules that are easy to get around. A good system that many people can use at the time needs to have strict rules about how much you can read, write and store and these rules need to be enforced by the system itself not by the users computer. Li and Zhu et al.[19,20] have talked about how to make cloud data management safe and trustworthy. Most of the time people do not think about how to make sure one user does not use up all the resources that are meant for everyone.

SHC has features like rate limiting and quota enforcement that use Redis, which means it can control how much each user can do. If you look at what people have written you can see that they have ideas about things like cryptography, blockchain, IPFS, access control and using machines to detect problems but nobody has put all of these things together into one system that is ready to use and has things like public auditability, unified cross-client authentication and pre-download threat evaluation.

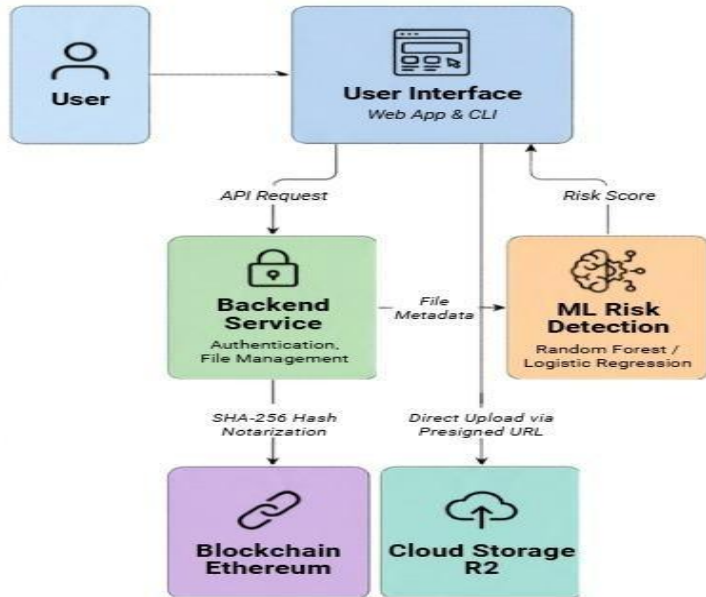
SHC is designed to fix all of these problems at the time using the best ideas from all of these areas of research by Tiwari and Nagpal et al. [23,24,25,26,27]SHC builds on the ideas, from these areas of research to make a system that is complete and ready to use .

Paper (Year)	Approach	Dataset/Benchmark	Key Result
Shrestha & Vassileva [1] (2018)	Blockchain incentive framework for research data owners	Research data sharing scenarios	Incentive model; third-party eliminated
Nakamoto [2] (2008)	Peer-to-peer electronic cash without intermediaries	Bitcoin network	Trustless decentralized consensus verified globally
Wood [9] (2014)	Ethereum smart contract platform	Ethereum mainnet	Programmable trustless transaction execution
Benet [8] (2014)	Content-addressed versioned P2P file system (IPFS)	IPFS network	No single point of failure; distributed availability
Shamir [10] (1979)	Secret sharing scheme splitting keys into n shares	Cryptographic key mgmt.	Threshold-based key reconstruction without full disclosure
Wu et al. [4] (2019)	Traceable attribute-based encryption for blockchain	Blockchain access control	Improved authenticity; decryption efficiency limited
Zhang & Zhao [5] (2018)	Smart contract-based digital rights management	License & transaction systems	Transparent license assignment; high computational cost
Ma et al. [6] (2018)	DRMchain dual blockchain for digital content protection	Protected digital content	High trust protection; Ethereum payments not addressed
Tiwari et al. [23] (2023)	Systematic review of ensemble ML for social network security	Cross-domain NLP datasets	Ensemble methods outperform single classifiers consistently
Tiwari et al. [24] (2022)	ML techniques for COVID-19 world effects prediction	Global COVID-19 datasets	Accurate predictions using ensemble classifiers
Tiwari et al. [25] (2024)	PSO-based Ensemble Fusion Voting Model	Five cross-domain datasets	99.42% training accuracy; 89.54% testing accuracy
Tiwari & Nagpal [26] (2022)	KEAHT attention-based hybrid transformer for sentiment	COVID-19 & farmer protest tweets	Improved contextual sentiment classification accuracy
Tiwari & Nagpal [27] (2023)	Blockchain-based security for social media computing	Social media data	Unified framework for decentralized social content security

III. METHODOLOGY

A. System Framework

The SHC platform is made up of different parts that work together. The backend service was written in Go. A web interface was built using Next.js and TypeScript.



System Components

Let: B denote the backend service (Go),

- W denote the web interface (Next.js, TypeScript),
- C denote the command-line client (Rust),
- S denote the storage layer (Cloudflare R2),
- M denote the machine learning risk analysis module

E denote the blockchain anchoring component

. Let the unified platform be defined as the tuple:

$$\mathcal{P} = (B, W, C, S, M, E)$$

1. User Authentication and Authorization

Let $\mathcal{U}$ denote the set of all users. Authentication is defined as the function

$$\text{Auth}: \mathcal{U} \times \text{Cred} \rightarrow \text{true, false}$$

For any user $u \in \mathcal{U}$, access via any client (W or C) is regulated by:

$\text{Perm}: \mathcal{U} \times \mathcal{A} \rightarrow \text{allow, deny}$ where $\mathcal{A}$ is the set of possible actions (upload, download, delete, etc.).

2. File Storage & Access

Let $\mathcal{F}$ be the set of all files, and each file $f \in \mathcal{F}$ has metadata $\mu(f)$.

Users interact with files via: Backend API (API_B)

Direct storage link generation ($\mathcal{L}: \mathcal{F} \rightarrow \mathcal{U} \rightarrow \mathbb{L}$), where $\mathbb{L}$ is the set of presigned R2 links

Total storage usage by a user u :

$$\text{Usage}(u) = \sum_{f \in \mathcal{F}_u} \text{size}(f) \leq \text{Quota}(u) \text{ where } \mathcal{F}_u \text{ is the set of files owned by } u, \text{ and } \text{Quota}(u) \text{ is the assigned storage quota.}$$

3. File Integrity via Blockchain Anchoring

For each uploaded file f , compute its SHA-256 hash: $h(f) = \text{SHA256}(f) \in 0,1^{256}$

Let

Record: $\mathcal{F} \rightarrow \mathcal{B}$ where $\mathcal{B}$ is the set of records on the blockchain E .

$\text{Record}(f) = \text{write}_E[\text{id}(f), h(f), t(f)]$ where $\text{id}(f)$ is a unique ID for f , $t(f)$ is the timestamp.

File integrity verification:

$$\text{Verify}(f') = \{\text{true, if } h(f') = \text{Record}(f) \text{ false, otherwise}$$

B. Data Collection

Our machine learning service uses a dataset because collecting real data is really hard and expensive. The synthetic data makes our models stronger by showing them different situations so they work better in the real world. We taught our forest classifier using 3200 examples with 14 things to look at and these examples include both good and bad ways that people share files. Our logistic regression pipeline was taught using 3200 text examples that show bad things people might say including words like verify, urgent and invoice and also things like when a file has two extensions. The information about the files like how big they're, how many times they have been downloaded, how often they are uploaded and what kind of file they are also helps teach our model. By looking at what's in the files and what the files are, like our model can give a better idea of how risky a file is when we manage files.

C. Data Preprocessing

The machine learning service needs features that're well organized and features that are based on text before it can classify things. The raw data came from the details of files and the attributes of share links. File size, Download count, How often files are uploaded, Request behavior are all features. The type of file, file extension and whether it can be seen are features. Before using text fields like file names, URLs and suspicious keywords they were cleaned up. This cleanup process involved: converting text to lowercase, removing symbols, breaking down text into individual words, getting rid of irrelevant words that do not add much value. Words that are often linked with phishing or malware were kept because they are indicators. To help the computer understand the text, TF-IDF, which looks at how a term appears and how unique it is, was used on file names and URLs.

D. Model Training

The system uses a kind of machine learning called dual model architecture. This is made using a dataset. The reason for this is that it is really hard to get real data.. Even if we get it we have to put all the data together which is also a big task.

The system has a Random Forest Classifier. This classifier makes 220 guesses. Goes up to 10 levels deep. It also gives weight to classes that have less data. We trained this classifier using 3200 samples. These samples have 14 features. We also used a function that knows about the area we are working in. We split our data into two parts. 80% Of the data is used for training. 20% Is used for testing. Our classifier is very good at its job. It got a score of 0.897. We also checked its stability by doing an experiment five times.

We also have another model. This model uses Logistic Regression and TF-IDF pipeline. This model is trained using 3000 text samples. These samples are either safe. They can cause harm. We added some words to these samples to make them look more real. This model helps us find out if a file has phishing language in it.

We saved both our models in files using a library called joblib. This way we can use them again without having to train them over again.

• Sigmoid Function

Synthetic label this generation uses the function to convert a weighted feature score into a probability between 0 and 1 for assigning class labels. Each feature in vector $\mathbf{x}$ is multiplied by a weight w , where larger weights represent stronger risk indicators

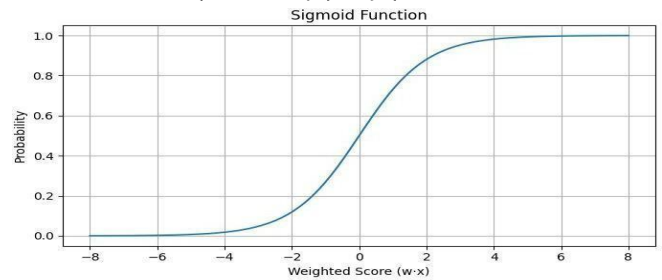
based on domain knowledge.

Given a feature vector $\mathbf{x} = (x_1, \dots, x_n)$ and weights $\mathbf{w} = (w_1, \dots, w_n)$:

$$f(\mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

where $\sigma(\cdot)$ is the sigmoid function, and b is the bias (intercept) term. This maps the weighted sum to a class probability:

$$P(\text{class} = 1 | \mathbf{x}) = f(\mathbf{x})$$



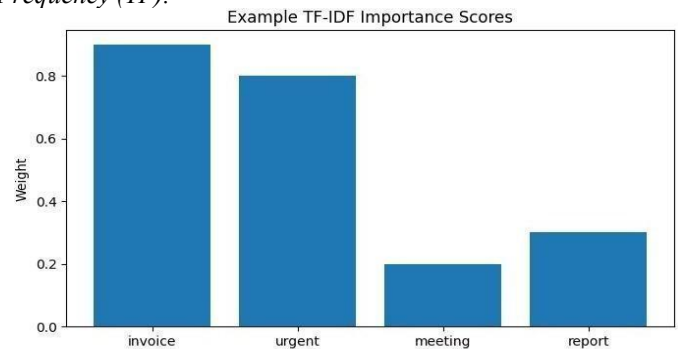
• TF-IDF (vectorization model)

It converts text into numerical features by measuring how important any in a document. TF (term frequency) counts how often the word appears in the document, while IDF (inverse document frequency) reduces the weight of very common words across all documents.

TF-IDF turns text documents into numerical vectors:

For term t in document d , across corpus D :

Term Frequency (TF):



$$TF(t, d) = \frac{\text{count}(t, d)}{\sum_t \text{count}(t, d)}$$

Inverse Document Frequency (IDF):

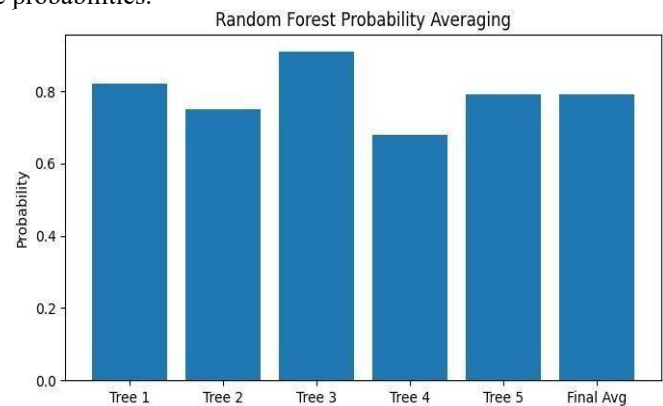
$$IDF(t, D) = \log \left(\frac{|D|}{1 + |\{d \in D : t \in d\}|} \right)$$

TF-IDF Weight

$$TF-IDF(t, d, D) = TF(t, d) \cdot IDF(t, D)$$

• Random Forest probability estimate

Random Forest predicts probability by combining the outputs of many decision trees in the forest. Each tree gives its own probability $p_t(\mathbf{x})$ for an input $\mathbf{x}$, and the final prediction is the average of all tree probabilities.



$$P(\text{class} = 1 | \mathbf{x}) = \frac{1}{T} \sum_{t=1}^T p_t(\mathbf{x})$$

E. Experimental Setup

All these experiments were done on a Windows machine with three services. The Machine Learning Risk Service uses Python 3 and FastAPI 0.116. Uvicorn to serve the risk scoring application programming interface on my local computer. The Backend application programming interface uses Go 1.24 and the Fiber v3 framework, PostgreSQL via GORM to store things Redis for caching and limiting how times things can happen and Cloudflare R2 as the object store that is similar to AWS S3.

- The Frontend is built with Next.js 14 React 18 and TypeScript. The Machine Learning libraries and what they do in the pipeline are as follows.
- NumPy (v2.1.3) :This thing helps us work with arrays in a way and it lets us build the feature matrix that we need to train our models. We can use it for both Random Forest and Logistic Regression pipelines.
- joblib (v1.5.1): used joblib to save our trained models to the disk. This way we can load them again fast when we need to make predictions without having to train them all over again
- SHAP (v0.46.0) :also used SHAP to understand how our models make predictions. It has used this algorithm called TreeExplainer that breaks down each prediction into smaller parts so we can see how each feature affects the result.

IV. Results & Discussion

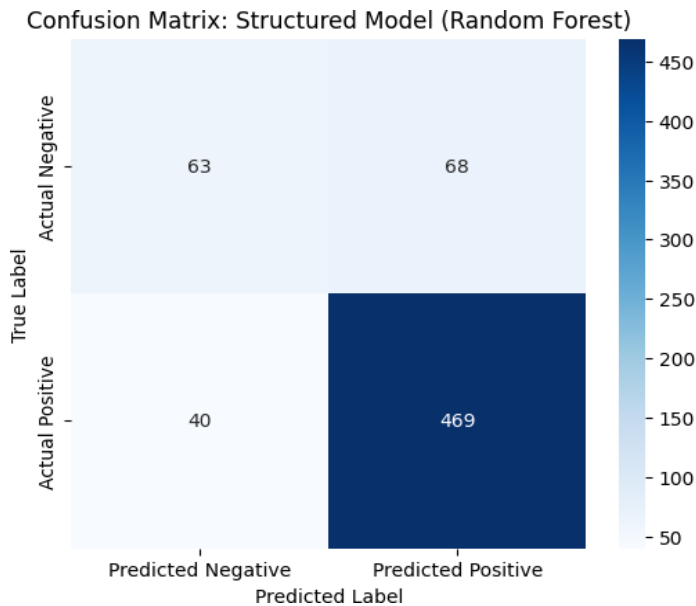
A. Comparative Result Analysis:

The SHC platform was compared to cloud file sharing systems and basic machine learning methods. The results show that SHC is more secure because it uses blockchain to verify files explainable AI to score risks allows users to upload files directly to storage and can be accessed by people through the web and command line interface. SHC uses a public blockchain to verify files, which means that any changes to uploaded files can be detected by outside parties rather than just relying on the platform to be trustworthy. This is done using SHA-256 notarization on the Ethereum Sepolia network.

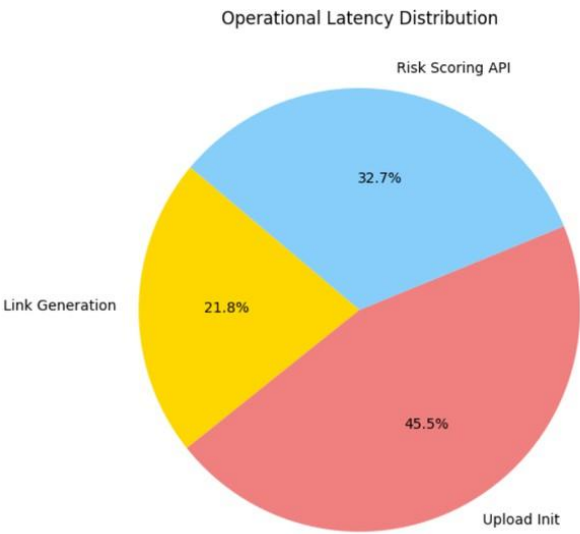
The SHC model was also compared to a Random Forest model. The SHC model did better in world tests because it combines analysis of file information with text screening using TF-IDF. The Random Forest model was good at classifying files. The SHC model was even better at detecting phishing attempts, suspicious links and fake file names because it also uses a Logistic Regression text pipeline.

The way SHC allows users to upload files directly to storage is also better than methods. Of sending large files through the server SHC gives users a special link to upload files directly to storage, which reduces bottlenecks and makes the system more scalable.

A. Risk Detection Performance



B. Upload Latency



C. Feature Importance (SHAP SUMMARY)

Metric	Value
Mean verification latency	~2.6 s
Verified hash match	Successful
Tamper detection	Successful
Verified score reduction	-25

D. Blockchain Verification Latency

Upload	Mean SHAP value	Risk Direction
MIME Mismatch	High	+Risk
Suspicious Keywords	High	+Risk
File Size	Medium	Bidirectional
Double Extension	Medium	+Risk

V. CONCLUSION & FUTURE SCOPE

This study is about SHC, a hybrid cloud file sharing platform. It was created to deal with the trust and safety issues that regular cloud storage systems have. The system they made combines a things: it can upload files directly to storage it uses blockchain to check if files are okay it uses machine learning to score risks it explains why it makes certain

decisions and it can be used on many platforms, including on the web and with commands.

The people who did this study tried it out. Got some good results. They found that it was very good, at telling what was good and what was bad it could find files and it gave consistent results when they tried it many times. The blockchain part was able to find files that had been tampered with which made it easier to trust the system. The part that explains things also helped, because it showed why some files were flagged as bad. So SHC shows that using blockchain and machine learning together can make a file sharing system that's smarter, safer and more trustworthy. Future work will focus on replacing the synthetic training data with real-world labeled datasets such as VirusTotal or EMBER to improve model generalizability. Planned enhancements include integrating federated learning to enable privacy-preserving model updates across users, extending blockchain verification to support decentralized storage proofs, and incorporating user feedback. Additionally, adversarial robustness testing and support for multi-modal file analysis (images, audio) are identified as key directions for improving detection coverage.

VI. REFERENCES

- [1] Shrestha, A.K.; Vassileva, J. Blockchain-Based Research Data Sharing Framework for Incentivizing the Data Owners. In *International Conference on Blockchain*; Springer: Cham, Switzerland, 2018; pp. 259–266.
- [2] Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008. Available online: <https://bitco.in/pdf/bitcoin.pdf> (accessed on 6 April 2019).
- [3] Crosby, M.; Pattanayak, P.; Verma, S.; Kalyanaraman, V. Blockchain technology: Beyond bitcoin. *Appl. Innov.* 2016, 2, 71.
- [4] Wu, A.; Zhang, Y.; Zheng, X.; Guo, R.; Zhao, Q.; Zheng, D. Efficient and privacy-preserving traceable attribute-based encryption in blockchain. *Ann. Telecommun.* 2019, 74, 401–411.
- [5] Zhang, Z.; Zhao, L. A Design of Digital Rights Management Mechanism Based on Blockchain Technology. In *International Conference on Blockchain*; Springer: Cham, Switzerland, 2018; pp. 32–46.
- [6] Ma, Z.; Jiang, M.; Gao, H.; Wang, Z. Blockchain for digital rights management. *Future Gener. Comput. Syst.* 2018, 89, 746–764.
- [7] Rowhani-Farid, A.; Allen, M.; Barnett, A.G. What incentives increase data sharing in health and medical research? A systematic review. *Res. Integr. Peer Rev.* 2017, 2, 4.
- [8] Benet, J. Ipfs-content addressed, versioned, p2p file system. *arXiv* 2014, arXiv:1407.3561.
- [9] Wood, G. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Proj. Yellow Pap.* 2014, 151, 1–32.
- [10] Shamir, A. How to share a secret. *Commun. ACM* 1979, 22, 612–613.
- [11] Nizamuddin, N.; Salah, K.; Azad, M.A.; Arshad, J.; Rehman, M.H. Decentralized document version control using ethereum blockchain and IPFS. *Comput. Electr. Eng.* 2019, 76, 183–197.
- [12] Hasan, H.R.; Salah, K. Proof of delivery of digital assets using blockchain and smart contracts. *IEEE Access* 2018, 6, 65439–65448.
- [13] Panescu, A.T.; Manta, V. Smart Contracts for Research Data Rights Management over the Ethereum Blockchain Network. *Sci. Technol. Libr.* 2018, 37, 235–245.
- [14] Chen, Y.; Li, H.; Li, K.; Zhang, J. An improved P2P file system scheme based on IPFS and Blockchain. In *Proceedings of the 2017 IEEE International Conference on Big Data (Big Data)*, Boston, MA, USA, 11–14 December 2017; pp. 2652–2657.
- [15] Steichen, M.; Fiz Pontiveros, B.; Norvill, R.; Shbair, W. Blockchain-Based, Decentralized Access Control for IPFS. In *Proceedings of the 2018 IEEE International Conference on Blockchain (Blockchain-2018)*, Halifax, NS, Canada, 30 July 2018–3 August 2018; pp. 1499–1506.
- [16] Sun, W.; Yu, S.; Lou, W.; Hou, Y.T.; Li, H. Protecting your right: Verifiable attribute-based keyword search with fine-grained owner-enforced search authorization in the cloud. *IEEE Trans. Parallel Distrib. Syst.* 2016, 27, 1187–1198.
- [17] Wang, S.; Zhang, Y.; Zhang, Y. A blockchain-based framework for data sharing with fine-grained access control in decentralized storage systems. *IEEE Access* 2018, 6, 38437–38450.
- [18] Park, J.S.; Youn, T.Y.; Kim, H.B.; Rhee, K.H.; Shin, S.U. Smart contract-based review system for an IoT data marketplace. *Sensors* 2018, 18, 3577.
- [19] Zhu, L.; Wu, Y.; Gai, K.; Choo, K.K.R. Controllable and trustworthy blockchain-based cloud data management. *Future Gener. Comput. Syst.* 2019, 91, 527–535.
- [20] Li, J.; Wu, J.; Chen, L. Block-secure: Blockchain based scheme for secure P2P cloud storage. *Inf. Sci.* 2018, 465, 219–231.
- [21] Madhumala, R. B.; Chhetri, S.; Akshatha, K. C., & Jain, H. (2021). Secure file storage & sharing on cloud using cryptography. *International Journal of Computer Science and Mobile Computing*, 10(5), 49.
- [22] Naz, M., Al-Zahrani, F. A., Khalid, R., Javaid, N., Qamar, M., Afzal, M. K., & Shafiq, M. (2019). A secure data sharing platform using blockchain and interplanetary file system. *Sustainability*, 11(24), 7054.
- [23] D. Tiwari, B. Nagpal, B. S. Bhati, A. Mishra, and M. Kumar, “A systematic review of social network sentiment analysis with comparative study of ensemble-based techniques,” *Artif. Intell. Rev.*, vol. 56, no. 11, pp. 13407–13461, 2023.
- [24] D. Tiwari, B. S. Bhati, F. Al-Turjman, and B. Nagpal, “Pandemic coronavirus disease (Covid-19): World effects analysis and prediction using machine-learning techniques,” *Expert Syst.*, vol. 39, no. 3, p. e12714, 2022.
- [25] D. Tiwari, B. Nagpal, B. S. Bhati, A. Mishra, and M. Kumar, “SPSO-EFVM: A Particle Swarm Optimization-based Ensemble Fusion Voting Model for Sentence-Level Sentiment Analysis,” *IEEE Access*, vol. 12, pp. 23707–23724, 2024.
- [26] D. Tiwari and B. Nagpal, “KEAHT: A Knowledge-Enriched Attention-Based Hybrid Transformer Model for Social Sentiment Analysis,” *New Gener. Comput.*, vol. 40, pp. 1165–1202, 2022.
- [27] D. Tiwari and B. Nagpal, *Blockchain Technology for Secure Social Media Computing*. Institution of Engineering and Technology, 2023..